REPORT (Part I)

The following sections present the decisions and respective inspirations that defined how we approached the **Knapsack Feasibility** problem.

Problem DSL

Design Choices:

Concepts like Target Weight (C) and Target Value (C) were merged directly into the KnapsackProblem concept, rather than becoming standalone classes.

We model the 0/1 Knapsack [GJ79]: each item can be selected at most once, so there is no item quantity (bounded knapsack [KPP04] is out of scope) and every decision variable needs a single bit.

Other concepts like Total Weight or Value and their respective constraints are derived during Transformation *T1* as linear expressions and atoms to keep the input problem DSL clean

Metamodel:

The metamodel is composed of only two classes: **KnapsackProblem** (the root problem container) and **Item** (the domain objects available for selection). These are connected in a `1..*` relationship, with all item instances being associated to a single KnapsackProblem.

All other concepts were therefore defined as attributes or derived values, ensuring no redundancy for a simpler metamodel design.

OCL Constraints:

Besides domain bound invariants like non-negative or strictly positive restraints for attributes like capacity, minimum value, weight and value, there is also a uniqueness constraint on the KnapsackProblem to ensure distinct item objects.

Solution DSL

All atoms were standardized to zero-comparisons to simplify the transformation into the quantum DSL. This normalization is important to how **Ripple-Carry** [VBE96, CDKM04] and **QFT adder** [Dra00] work.

Carrying **bitWidth** explicitly on the `Unknown` concept allows Transformation **T2** to directly calculate quantum register allocations in Qiskit without re-parsing variable bounds.

Algorithmic execution parameters like `AdderKind`, `iterations` and `shots` are isolated to keep the mathematical problem definition decoupled from backend execution choices. `iterations` and `shots` are the knobs of the Grover search [Gro96, BBHT98] that T2/T3 will build on top of the constraint system. In Part II, the required configuration model also explicitly carries the shared expression-register bit width and two's-complement encoding; transformations validate and consume these values rather than choosing defaults.

Metamodel:

The `Unknown` class is a domain-agnostic mathematical variable representing the KnapsackProblem decision variables, always with a unique name.

While `ConstraintSystem` serves as the root container for the entire mathematical model, the `Formula` is intended to be the abstract base class that can serve as the root for boolean expressions being referenced by the `ConstraintSystem`.

OCL Constraints:

To follow design by contract principles, class invariants were used to ensure the model is sound and suitable for quantum compilation.

Variable integrity constraints are required for decision variable name uniqueness and positive bitWidth registers.

Structural invariants for formula nodes and junction operands ensure a proper built abstract syntax tree.

Search space was also limited with a 20-qubit upper bound to prevent memory overflow in validation testing.

References

- [GJ79] M. Garey and D. Johnson. Computers and Intractability – A Guide to the Theory of NP-Completeness. Freeman and Company, New York, 1979.
- [KPP04] H. Kellerer, U. Pferschy, D. Pisinger. Knapsack Problems. Springer, 2004.
- [Gro96] L. Grover. “A fast quantum mechanical algorithm for database search”, STOC, 1996.
- [BBHT98] M. Boyer, G. Brassard, P. Høyer, A. Tapp. “Tight bounds on quantum searching”, Fortschritte der Physik, 1998.
- [VBE96] V. Vedral, A. Barenco, A. Ekert. “Quantum networks for elementary arithmetic operations”, Physical Review A, 1996.
- [CDKM04] S. Cuccaro et al., “A new quantum ripple-carry addition circuit”, arXiv:quant-ph/0410184, 2004.
- [Dra00] T. Draper. “Addition on a quantum computer”, arXiv:quant-ph/0008033, 2000.
- Assignment: "TACS 2627 Assignment, QMDE